

Trabajo Práctico 2 — JAVA

[7507/9502] Algoritmos y Programación III

Grupo 4

Primer cuatrimestre de 2026

Alumno	Número de padrón	Email
Altamirano, Adrian	111858	aaltamiranoc@fi.uba.ar
Cavieres, Ignacio	112966	icavieres@fi.uba.ar
Mena, Paul	110893	pmena@fi.uba.ar
Sanchez, Arturo	112759	asanchezc@fi.uba.ar

Índice

1. Introducción	2
2. Supuestos	2
3. Diagramas de clase	3
4. Diagrama de Secuencia	4
5. Detalles de Implementación	5
5.1. Patrones de Diseño Utilizados	5
5.2. Justificación de Herencia vs. Delegación	5
5.3. Principios de Diseño Aplicados (SOLID)	5

1. Introducción

El presente informe reúne la documentación de la solución del Trabajo Práctico 2: "Mafia" de la materia Algoritmos y Programación III. El objetivo principal consiste en desarrollar una aplicación basada en este juego de deducción social por turnos, aplicando todos los conceptos del paradigma de la orientación a objetos vistos en el curso.

Para la construcción de este sistema se utilizará un lenguaje de tipado estático y se trabajará activamente con metodologías de Desarrollo Guiado por Pruebas (TDD) e Integración Continua. En esta instancia inicial, correspondiente a la Entrega 0, el documento expone el modelado tentativo del dominio, detallando la estructura estática mediante diagramas de clases, el comportamiento dinámico de la Fase Nocturna a través de diagramas de secuencia y los supuestos asumidos para la resolución del problema.

2. Supuestos

A partir del análisis de los requerimientos y la construcción del modelo tentativo, el equipo ha adoptado los siguientes supuestos para delimitar el alcance del dominio:

- **Mecanismo de desempate configurable:** Aunque el sistema admite tanto el ballotage como la declaración de ronda sin eliminados en caso de empate, asumimos que la elección de este mecanismo no tiene un valor por defecto fijo, sino que se define estrictamente durante la configuración inicial de la partida antes de comenzar a jugar.
- **Consenso de la Mafia:** Las reglas establecen que la Mafia debe elegir a su víctima en conjunto y que el Padrino posee prioridad en su voto. Asumimos que el debate y la deliberación entre ellos ocurre fuera del alcance directo de nuestro modelo (por ejemplo, a través del chat del juego), y que el sistema únicamente recibe y registra la decisión final unificada para ejecutar el ataque.
- **Roles opcionales y turnos vacíos:** Dado que roles como el Detective y el Médico son opcionales según la configuración del mazo, asumimos que si no están presentes en la partida, o si el jugador que los poseía ya fue eliminado, el sistema simplemente saltea su turno durante la Fase Nocturna de manera automática y transparente, pasando a la siguiente acción sin interrumpir el flujo.
- **Votos en blanco o abstenciones:** Respecto a la abstención durante la Fase Diurna asumimos que el sistema directamente descarta a ese jugador del conteo. La eliminación por mayoría simple se calcula evaluando únicamente los votos válidos que sí fueron emitidos hacia los jugadores nominados.
- **El Moderador como sistema:** Dado que el rol de Moderador es asumido por la aplicación, asumimos que no es necesario modelar una carta ni un objeto **Jugador** particular para este fin. La propia orquestación del estado y de las fases (transiciones entre día y noche) se encargará de "narrar." exponer los eventos de forma automática a través de la interfaz.

3. Diagramas de clase

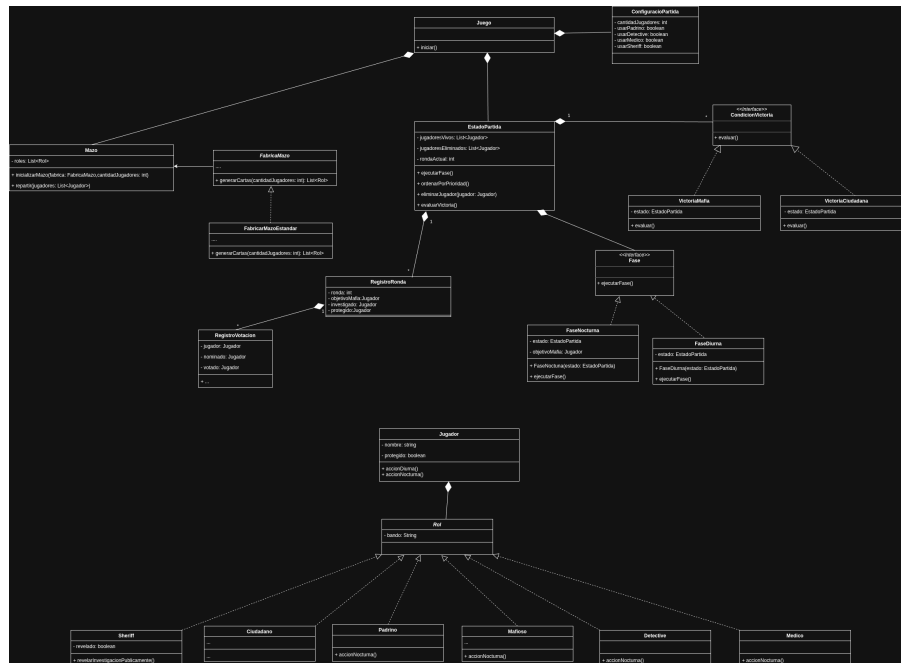
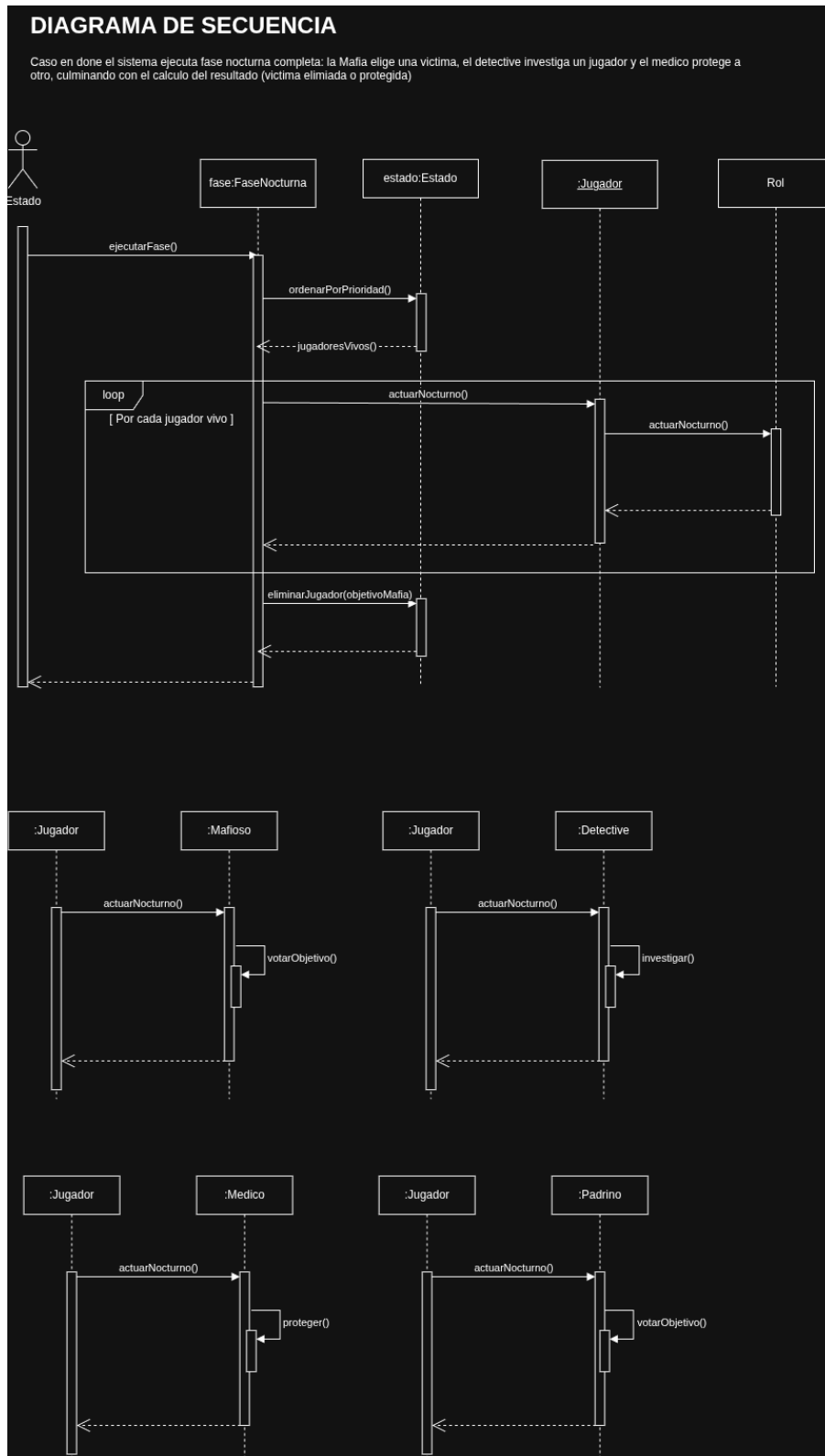


Figura 1:

4. Diagrama de Secuencia



5. Detalles de Implementación

5.1. Patrones de Diseño Utilizados

- **Strategy (Condiciones de Victoria):** Se implementó la interfaz genérica `CondicionVictoria` para evaluar el estado del juego. Dado que la partida verifica si alguien ganó al final de cada fase (Nocturna o Diurna), este patrón evita anidar múltiples estructuras condicionales en el `EstadoPartida` y permite que el sistema responda dinámicamente si ganan los Ciudadanos o la Mafia, manteniendo el código abierto a futuras expansiones.
- **Abstract Factory / Factory Method (Mazo de Cartas):** Se utilizó `FabricaMazo` para aislar la instanciación de los roles. La composición exacta del mazo (cantidad de mafiosos, ciudadanos y roles especiales) varía estructuralmente según la cantidad de jugadores ingresados al inicio de la partida. Delegar esto a una fábrica mantiene la clase `Mazo` enfocada únicamente en contener y repartir las cartas.
- **State (Fases del Juego):** Se abstrajo el comportamiento de la partida mediante la interfaz `Fase` (`FaseNocturna` y `FaseDiurna`). Esto resuelve el conflicto de la alternancia de turnos, permitiendo que `EstadoPartida` delegue la ejecución del momento actual sin tener que preguntar constantemente en qué momento del día se encuentra.

5.2. Justificación de Herencia vs. Delegación

- **Delegación (Jugador y Rol):** Se optó por la composición/delegación donde un `Jugador` contiene un `Rol`. Se descartó la herencia (ej. `JugadorMafioso` heredando de `Jugador`) porque el rol es una carta secreta asignada y ocultada durante la partida. Delegar el comportamiento nocturno al objeto `Rol` protege la identidad del jugador y mantiene la entidad `Jugador` libre de la lógica específica de las habilidades.

5.3. Principios de Diseño Aplicados (SOLID)

- **Principio de Responsabilidad Única (SRP) en RegistroRonda:** Uno de los puntos más conflictivos era el manejo del historial para asegurar la transparencia de la partida y el resumen final. En lugar de sobrecargar la clase `EstadoPartida`, se creó `RegistroRonda` para que asuma la responsabilidad exclusiva de almacenar los objetivos nocturnos y las votaciones diurnas de esa iteración específica.
- **Principio de Segregación de Interfaces (ISP) en Roles:** No todos los roles actúan de noche (ej. el Ciudadano o el Sheriff). Forzar un método genérico de acción nocturna en la clase base `Rol` habría roto este principio. Por ello, las habilidades específicas se manejan asegurando que solo los roles pertinentes deban resolver una acción en la `FaseNocturna`.